

- **NAME** : SIDDHESH SARAVADE
- **E.N.** – ADT23SOCB1120
- PTC BATCH 6

CineStream Backend Database Management System

1. Title

Design and Implementation of CineStream Backend Database Using SQL

2. Objective

The objective of this project is to design and implement a relational database management system for CineStream, an online movie streaming platform. The database centralizes movie catalog information, user account details, and movie viewing history while supporting daily operations and business analytics.

3. Problem Statement

CineStream currently manages movie data, user records, and viewing history using multiple spreadsheets. This causes data duplication, inconsistency, slow reporting, and difficulty in tracking user behavior. To overcome these issues, a relational database system is required.

4. Software Requirements

- MySQL Database Server
 - MySQL Workbench / phpMyAdmin
 - SQL Language
 - Windows/Linux Operating System
-

5. Database Design

Movies Table

Field Name	Data Type	Constraints
movie_id	INT	Primary Key, Auto Increment
title	VARCHAR(100)	NOT NULL
genre	VARCHAR(50)	NOT NULL
release_year	INT	-
rental_rate	DECIMAL(5,2)	-

Users Table

Field Name	Data Type	Constraints
user_id	INT	Primary Key, Auto Increment
username	VARCHAR(50)	UNIQUE, NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
join_date	DATE	-
status	VARCHAR(10)	Active/Inactive

WatchLog Table

Field Name	Data Type	Constraints
log_id	INT	Primary Key, Auto Increment
user_id	INT	Foreign Key
movie_id	INT	Foreign Key
watch_date	DATETIME	-
duration_minutes	INT	-

6. SQL Implementation

Create Database

```
CREATE DATABASE CineStreamDB;  
USE CineStreamDB;
```

Create Movies Table

```
CREATE TABLE Movies (  
    movie_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(100) NOT NULL,  
    genre VARCHAR(50) NOT NULL,  
    release_year INT,
```

```
        rental_rate DECIMAL(5,2)
    );
```

Create Users Table

```
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    join_date DATE,
    status VARCHAR(10)
);
```

Create WatchLog Table

```
CREATE TABLE WatchLog (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    movie_id INT,
    watch_date DATETIME,
    duration_minutes INT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (movie_id) REFERENCES Movies(movie_id)
);
```

7. Sample Data Insertion

Insert Movie

```
INSERT INTO Movies
(title, genre, release_year, rental_rate)
VALUES
('Interstellar', 'Sci-Fi', 2014, 4.99);
```

Insert Users

```
INSERT INTO Users
(username, email, join_date, status)
VALUES
('johndoe123', 'john@gmail.com', '2025-02-15', 'Inactive'),
('alice01', 'alice@gmail.com', '2025-03-10', 'Active');
```

Insert Watch Logs

```
INSERT INTO WatchLog
(user_id, movie_id, watch_date, duration_minutes)
VALUES
(1, 1, '2026-06-10 18:30:00', 120);
```

8. CRUD Operations

Create

```
INSERT INTO Movies
(title, genre, release_year, rental_rate)
VALUES
('Interstellar', 'Sci-Fi', 2014, 4.99);
```

Read

```
SELECT *
FROM Users
WHERE join_date > '2025-01-01'
AND status='Active';
```

Update

```
UPDATE Users
SET status='Active'
WHERE username='johndoe123';
```

Delete

```
DELETE FROM Users
WHERE status='Inactive'
AND user_id NOT IN
(
SELECT user_id
FROM WatchLog
);
```

9. JOIN Query for Genre Popularity Report

```
SELECT
m.genre AS Genre_Name,
COUNT(w.log_id) AS Total_Watch_Count
FROM Movies m
INNER JOIN WatchLog w
ON m.movie_id = w.movie_id
GROUP BY m.genre
ORDER BY Total_Watch_Count DESC;
```

Purpose

This query identifies the most popular movie genres by calculating how many times movies from each genre have been watched.

10. Expected Output

Genre Name	Total Watch Count
Action	20
Sci-Fi	15
Comedy	10
Drama	8

The results are displayed in descending order of popularity.

11. Advantages of the Database

1. Eliminates data redundancy.
 2. Maintains data integrity through primary and foreign keys.
 3. Supports efficient CRUD operations.
 4. Enables business analytics and reporting.
 5. Improves scalability for future growth.
 6. Provides accurate tracking of user viewing behavior.
-

12. Conclusion

The CineStream Backend Database successfully centralizes movie, user, and viewing information into a structured relational database. The system supports CRUD operations, maintains relationships through foreign keys, and generates analytical reports using SQL JOIN and GROUP BY operations. This database improves operational efficiency, data consistency, and decision-making capabilities for CineStream.